

HW2

December 2, 2025

1 HW2 Vibration

1.1 Group Members

Boyan Zhang - 999015290

Ziwa Yang - 999021173

Qinrong Yu - 999021223

Zhuorui Yun - 999021264

Rui Zhong - 99021405

period $T = 0.5$ s, the fundamental circular frequency is:

$$\omega_0 = \frac{2\pi}{T} = 4\pi \approx 12.57 \text{ rad/s}$$

Transfer Function

$$H(\omega) = \frac{1}{k - m\omega^2 + j(c\omega)}$$

Where:

$$k = m\omega_n^2$$

$$c = 2\zeta m\omega_n$$

We calculate the response $x(t)$ for each harmonic separately and sum them up

$$x(t) = \sum_{n=1}^4 2 \cdot \text{Re}(F_n H(n\omega_0) e^{jn\omega_0 t})$$

```
[7]: import numpy as np
import sympy as sy
import matplotlib.pyplot as plt

t = sy.symbols('t', real = True)

Wn = 36
damping_ratio = 0.008
T = 0.5
m = 500
```

```

Wr0 = 2*sy.pi/T * 0
Wr1 = 2*sy.pi/T * 1
Wr2 = 2*sy.pi/T * 2
Wr3 = 2*sy.pi/T * 3
Wr4 = 2*sy.pi/T * 4

F0 = 0
F1 = 3 + 1j * 2.1
F2 = -0.6 + 1j * 0.5
F3 = 0.84 - 1j * 0.75
F4 = 0.01 - 1j * 0.03

X0 = (F0/m)/((Wn**2)-(Wr0**2) + (1j * 2 * damping_ratio * Wn * Wr0))
X1 = (F1/m)/((Wn**2)-(Wr1**2) + (1j * 2 * damping_ratio * Wn * Wr1))
X2 = (F2/m)/((Wn**2)-(Wr2**2) + (1j * 2 * damping_ratio * Wn * Wr2))
X3 = (F3/m)/((Wn**2)-(Wr3**2) + (1j * 2 * damping_ratio * Wn * Wr3))
X4 = (F4/m)/((Wn**2)-(Wr4**2) + (1j * 2 * damping_ratio * Wn * Wr4))

x_expr = X0 * sy.exp(1j * Wr0 * t) + \
          X1 * sy.exp(1j * Wr1 * t) + \
          X2 * sy.exp(1j * Wr2 * t) + \
          X3 * sy.exp(1j * Wr3 * t) + \
          X4 * sy.exp(1j * Wr4 * t) + \
          sy.conjugate(X0) * sy.exp(-1j * Wr0 * t) + \
          sy.conjugate(X1) * sy.exp(-1j * Wr1 * t) + \
          sy.conjugate(X2) * sy.exp(-1j * Wr2 * t) + \
          sy.conjugate(X3) * sy.exp(-1j * Wr3 * t) + \
          sy.conjugate(X4) * sy.exp(-1j * Wr4 * t)

print('X =')
x_expr

```

X =

$$\begin{aligned}
[7]: & \frac{(0.006 + 0.0042i)e^{4.0i\pi t}}{-16.0\pi^2 + 1296 + 2.304i\pi} + \frac{(-0.0012 + 0.001i)e^{8.0i\pi t}}{-64.0\pi^2 + 1296 + 4.608i\pi} + \frac{(0.00168 - 0.0015i)e^{12.0i\pi t}}{-144.0\pi^2 + 1296 + 6.912i\pi} + \\
& \frac{(2.0 \cdot 10^{-5} - 6.0 \cdot 10^{-5}i)e^{16.0i\pi t}}{-256.0\pi^2 + 1296 + 9.216i\pi} + \frac{(2.0 \cdot 10^{-5} + 6.0 \cdot 10^{-5}i)e^{-16.0i\pi t}}{-256.0\pi^2 + 1296 - 9.216i\pi} + \frac{(0.00168 + 0.0015i)e^{-12.0i\pi t}}{-144.0\pi^2 + 1296 - 6.912i\pi} + \\
& \frac{(-0.0012 - 0.001i)e^{-8.0i\pi t}}{-64.0\pi^2 + 1296 - 4.608i\pi} + \frac{(0.006 - 0.0042i)e^{-4.0i\pi t}}{-16.0\pi^2 + 1296 - 2.304i\pi}
\end{aligned}$$

```

[8]: f_expr = F0 * sy.exp(1j * Wr0 * t) + \
      F1 * sy.exp(1j * Wr1 * t) + \
      F2 * sy.exp(1j * Wr2 * t) + \
      F3 * sy.exp(1j * Wr3 * t) + \
      F4 * sy.exp(1j * Wr4 * t) + \
      sy.conjugate(F0) * sy.exp(-1j * Wr0 * t) + \
      sy.conjugate(F1) * sy.exp(-1j * Wr1 * t) + \

```

```

sy.conjugate(F2) * sy.exp(-1j * Wr2 * t) + \
sy.conjugate(F3) * sy.exp(-1j * Wr3 * t) + \
sy.conjugate(F4) * sy.exp(-1j * Wr4 * t)
print('f(t)=')
f_expr

```

f(t)=

[8]: $(3.0 + 2.1i)e^{4.0i\pi t} + (-0.6 + 0.5i)e^{8.0i\pi t} + (0.84 - 0.75i)e^{12.0i\pi t} + (0.01 - 0.03i)e^{16.0i\pi t} + (0.01 + 0.03i)e^{-16.0i\pi t} + (0.84 + 0.75i)e^{-12.0i\pi t} + (-0.6 - 0.5i)e^{-8.0i\pi t} + (3.0 - 2.1i)e^{-4.0i\pi t}$

```

[4]: import numpy as np
import matplotlib.pyplot as plt

m = 500.0          # Mass in kg
wn = 36.0          # Natural freq in rad/s
zeta = 0.008       # Damping ratio
T = 0.5            # Period in seconds

w0 = (2 * np.pi) / T # Fundamental frequency (rad/s)
k = m * (wn**2)       # Stiffness (N/m)
c = 2 * zeta * m * wn # Damping coefficient (N*s/m)

F_coeffs_kN = [
    3 + 2.1j,      # F1
    -0.6 + 0.5j,   # F2
    0.84 - 0.75j,  # F3
    0.01 - 0.03j   # F4
]

F_amps = np.array(F_coeffs_kN) * 1000

t = np.linspace(0, 2*T, 1000)

f_total = np.zeros_like(t)
x_total = np.zeros_like(t)

print(f"System Natural Frequency: {wn:.2f} rad/s")
print("-" * 60)
print(f"{'Harmonic':<10} | {'Freq (rad/s)':<15} | {'Force (N)':<15} | {'Disp_↵(m)':<15}")
print("-" * 60)

for n, Fn in enumerate(F_amps, start=1):
    omega = n * w0

```

```

# Calculate Receptance H(omega)
#  $H(\omega) = 1 / (k - m\omega^2 + jcw)$ 
H_w = 1 / (k - m*(omega**2) + 1j*c*omega)

# Calculate Response Coefficient Xn
Xn = Fn * H_w

# --- Time Domain Reconstruction ---
# We multiply by 2 * Real part to account for both positive
# and negative harmonics (rendering a real result).

# Force component
f_n_t = 2 * np.real(Fn * np.exp(1j * omega * t))
f_total += f_n_t

# Displacement component
x_n_t = 2 * np.real(Xn * np.exp(1j * omega * t))
x_total += x_n_t

print(f"{n:<10} | {omega:<15.2f} | {np.abs(Fn):<15.2f} | {np.abs(Xn):<15.2f}")

plt.figure(figsize=(10, 6))

plt.plot(t, f_total / 1000, 'b', label='Excitation Force $f(t)$ [kN]',
         linewidth=1.5)

plt.plot(t, x_total * 100, 'r--', label='Response $100 \times x(t)$ [m]',
         linewidth=2)

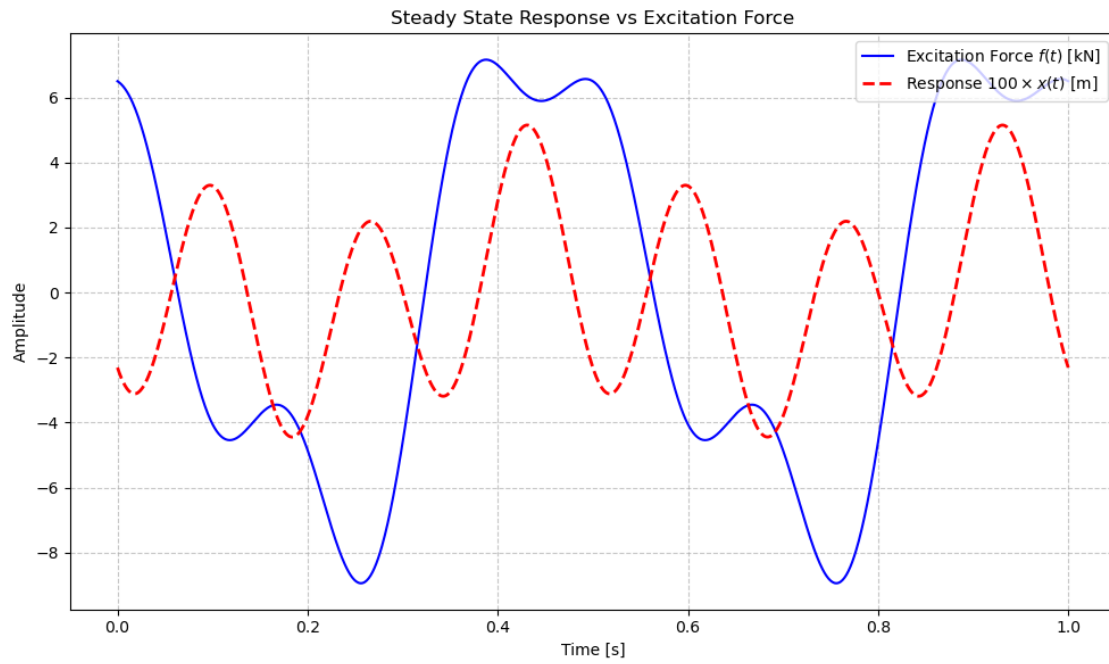
plt.title('Steady State Response vs Excitation Force')
plt.xlabel('Time [s]')
plt.ylabel('Amplitude')
plt.grid(True, which='both', linestyle='--', alpha=0.7)
plt.legend(loc='upper right')
plt.tight_layout()

plt.show()

```

System Natural Frequency: 36.00 rad/s

Harmonic	Freq (rad/s)	Force (N)	Disp (m)
1	12.57	3661.97	0.0064
2	25.13	781.02	0.0024
3	37.70	1126.10	0.0177



1.2 Explanation for the diagram

For the blue line (Force), the amplitude of the 1st harmonic $|3 + 2.1j| \approx \mathbf{3.66}$ kN is very large.

For red (output), the frequency of 3rd harmonic is close to resonance frequency, so it is dominant by 3rd harmonic.